Deployment of Keycloak

VirtualBox, Docker, PostgreSQL, and NGINX Environment

Prepared by: Ihor Zadorozhniak

Deployment option: Local virtual machine in Oracle VirtualBox

1. Purpose of the Laboratory Work

The purpose of this laboratory work is to prepare a local virtualized environment for deploying Keycloak as a centralized identity and access management platform. The completed environment will include an Ubuntu virtual machine, Docker Engine, a PostgreSQL database, a Keycloak container, and NGINX as an HTTPS reverse proxy.

2. Selected Deployment Method

The local deployment option was selected. The server is hosted in Oracle VirtualBox and uses a bridged network adapter so that the guest virtual machine can operate as a separate device on the same local network as the host computer.

3.1 Creation and Startup of the Virtual Machine

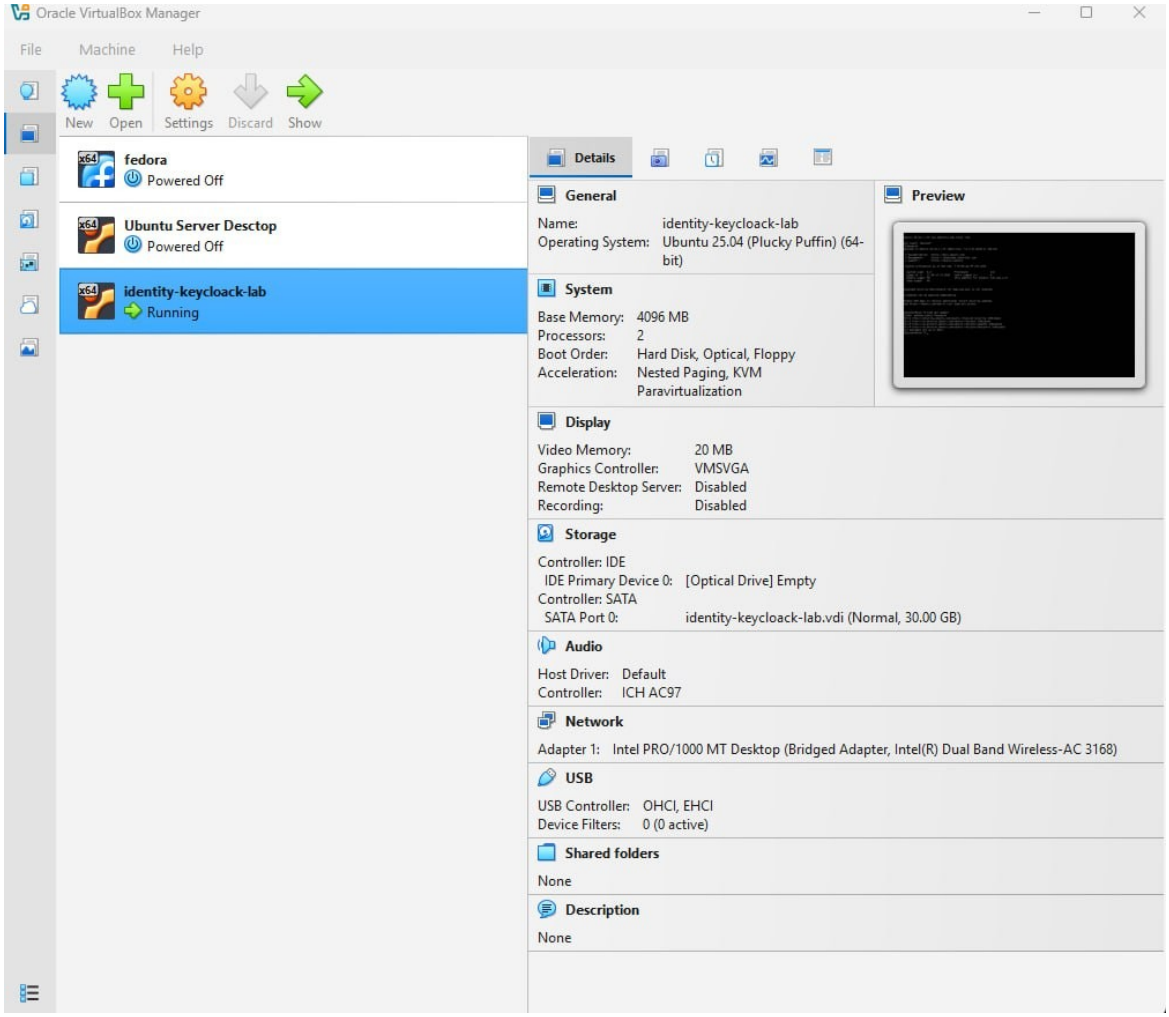


Figure 1 - The identity-keycloak-lab virtual machine running in Oracle VirtualBox

An Oracle VirtualBox virtual machine named identity-keycloak-lab was created and successfully started for the local deployment option. The configuration shown allocates 4,096 MB of RAM, two virtual processors, and a 30 GB virtual disk, which provides the resources required for the laboratory services. VirtualBox currently displays Ubuntu 25.04 as the operating-system profile; because the laboratory requires Ubuntu 24.04 LTS, the installed guest version will be verified later with the cat /etc/os-release command.

3.2 Configuration of the Virtual Network Adapter

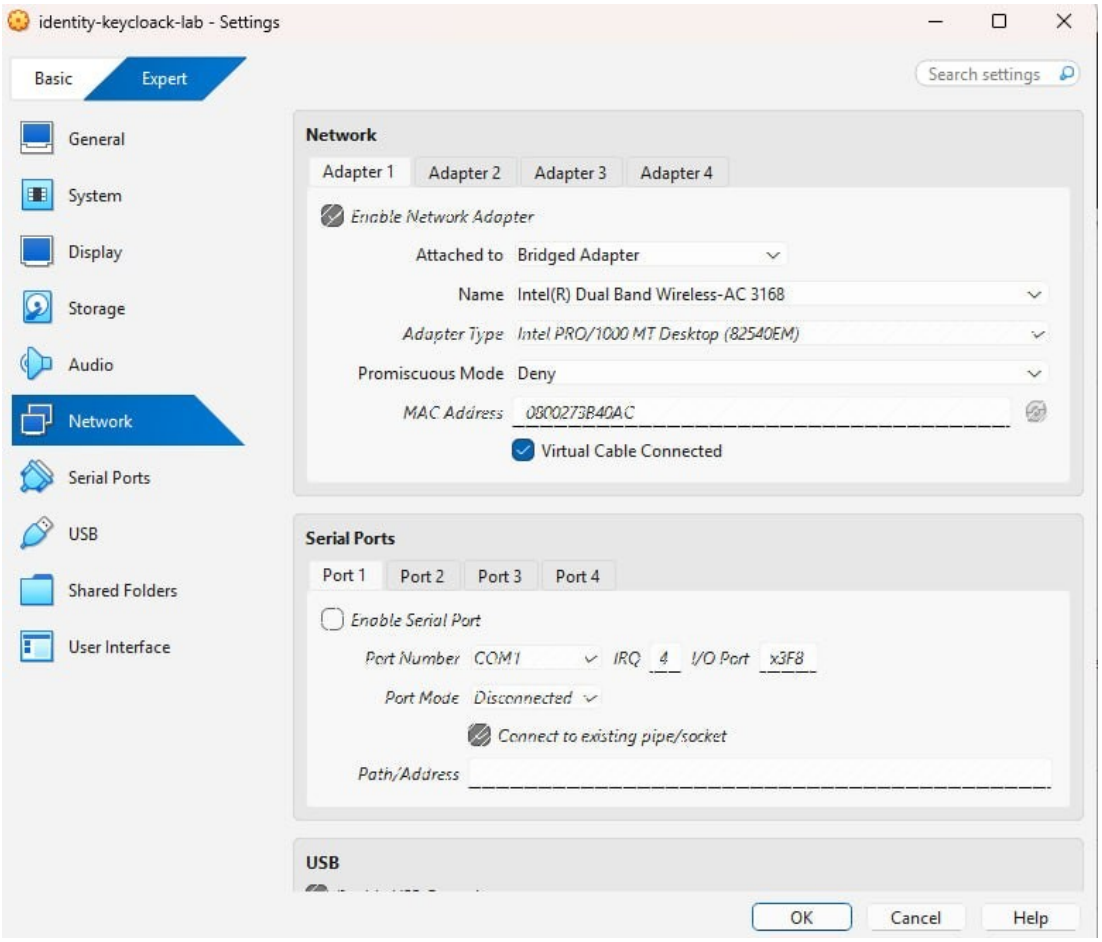


Figure 2 - Bridged Adapter configuration for the virtual machine

The first virtual network adapter was enabled and attached in Bridged Adapter mode to the Intel(R) Dual Band Wireless-AC 3168 interface. The virtual cable is connected and promiscuous mode remains set to Deny, allowing the VM to obtain an address from the local network while preserving the recommended laboratory settings.

3.3 Allocation of Processor Resources

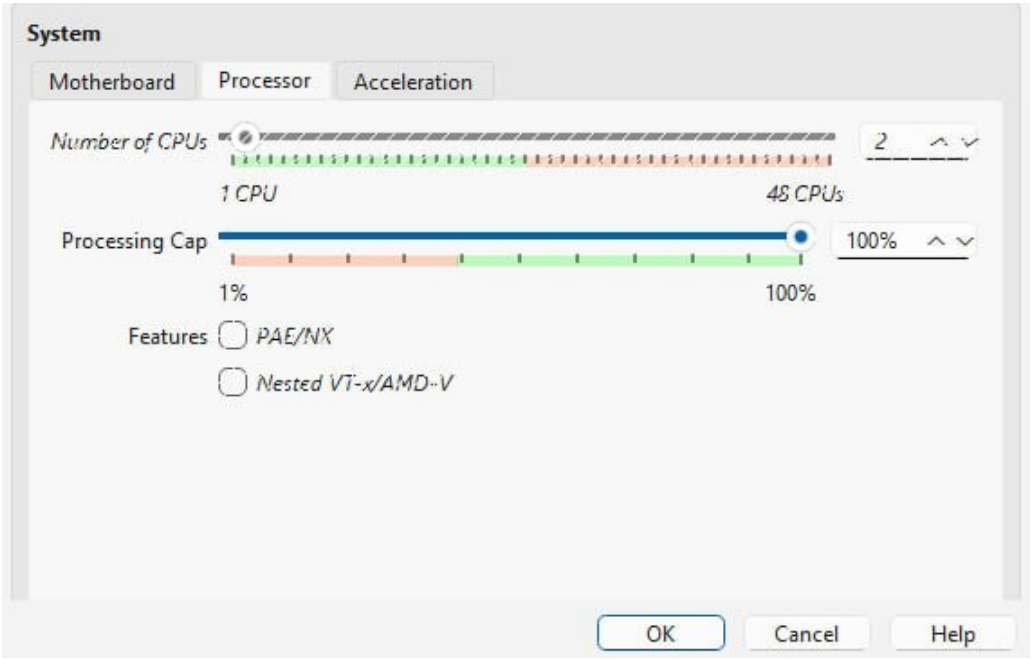


Figure 3 - Processor configuration of the identity-keycloak-lab virtual machine

The virtual machine was configured with two virtual CPUs and a 100% execution cap. This allocation satisfies the laboratory requirement and provides adequate processing capacity for running Ubuntu together with the PostgreSQL, Keycloak, and NGINX containers.

3.4 SSH Connection and Hostname Verification

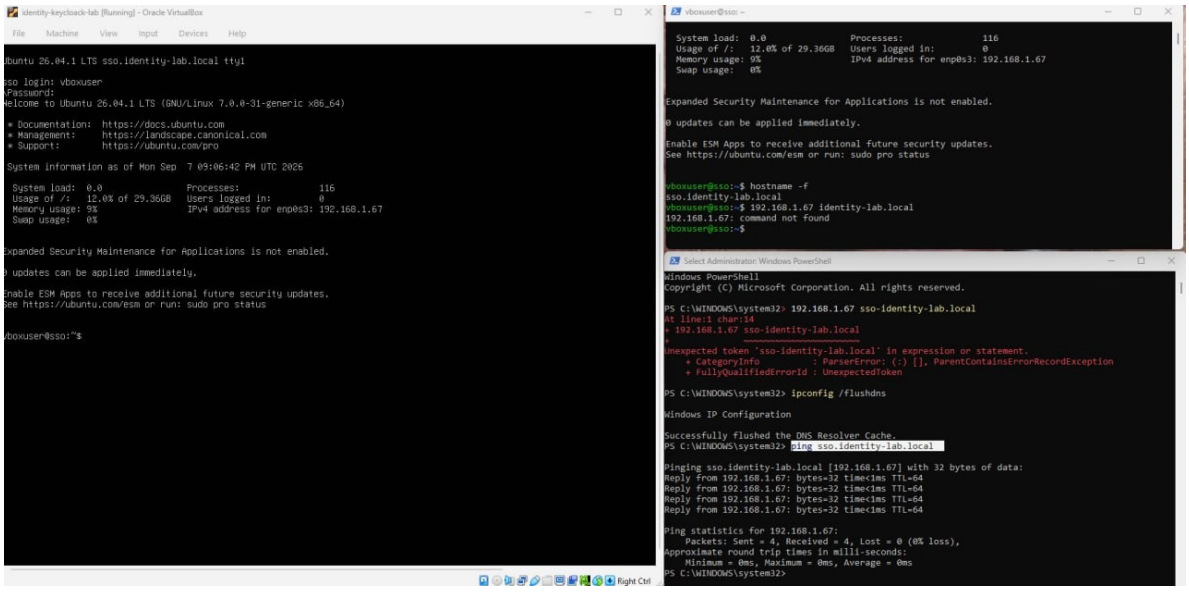


Figure 4 - SSH access to the VM and verification of hostname resolution

An SSH terminal session was established from the Windows host to the virtual machine, which received the IPv4 address 192.168.1.67 on interface enp0s3. The hostname -f command confirmed the fully qualified domain name sso.identity-lab.local; after the Windows DNS resolver cache was flushed, a ping test successfully resolved the hostname to the same address and completed with 0% packet loss. The VM console reports Ubuntu 26.04.1 LTS, which differs from the Ubuntu 24.04 LTS version specified in the laboratory instructions and should therefore be corrected or approved before the remaining deployment stages are completed.

4. Docker Engine Installation and Verification

Docker Engine was installed from the official Docker repository. The following evidence documents the repository preparation, package installation, service validation, functional test, and configuration of non-root access.

4.1 Installation of Prerequisite Packages

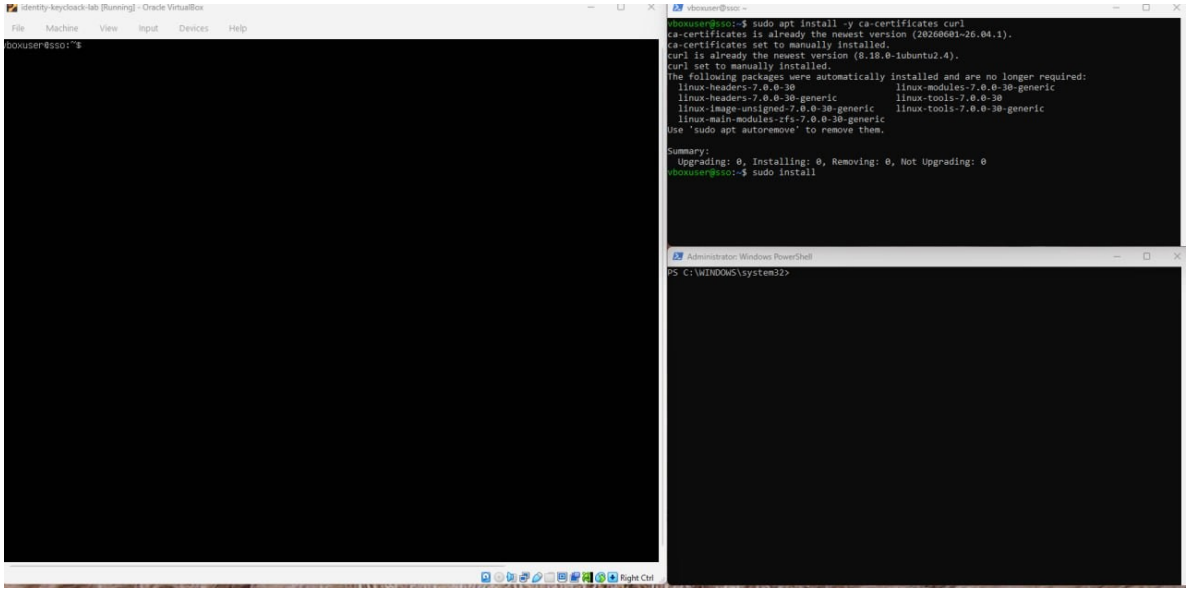


Figure 5 - Installation check for the ca-certificates and curl packages

The package manager was used to install ca-certificates and curl, which are required for secure access to the official Docker repository and its signing key. The output confirms that both packages were already installed at their newest available versions, so no additional package changes were necessary.

4.2 Addition of the Official Docker Signing Key

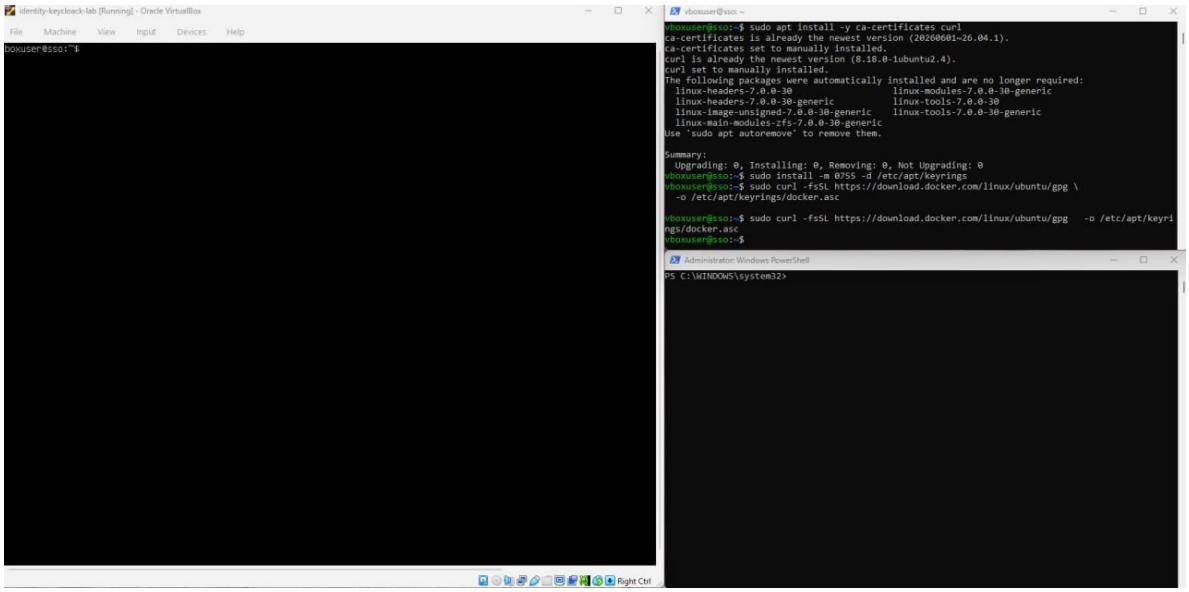


Figure 6 - Creation of the APT keyring directory and download of the Docker GPG key

The /etc/apt/keyrings directory was prepared and the official Docker GPG key was downloaded to /etc/apt/keyrings/docker.asc. Read permissions were then applied to the key file so that APT could use it to verify the authenticity and integrity of Docker packages.

4.3 Configuration of the Docker APT Repository

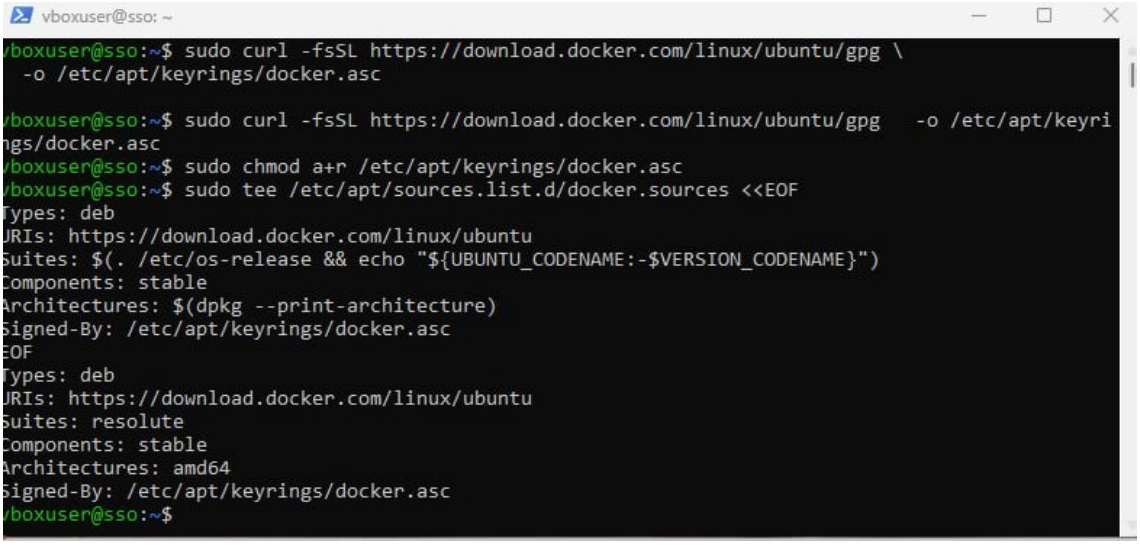


Figure 7 - Docker repository configuration stored in docker.sources

The official Docker repository was added in /etc/apt/sources.list.d/docker.sources with the stable component, the system architecture, and the downloaded signing key. The displayed configuration confirms that APT will retrieve Docker packages from download.docker.com and validate them using docker.asc.

4.4 Installation of Docker Engine Components

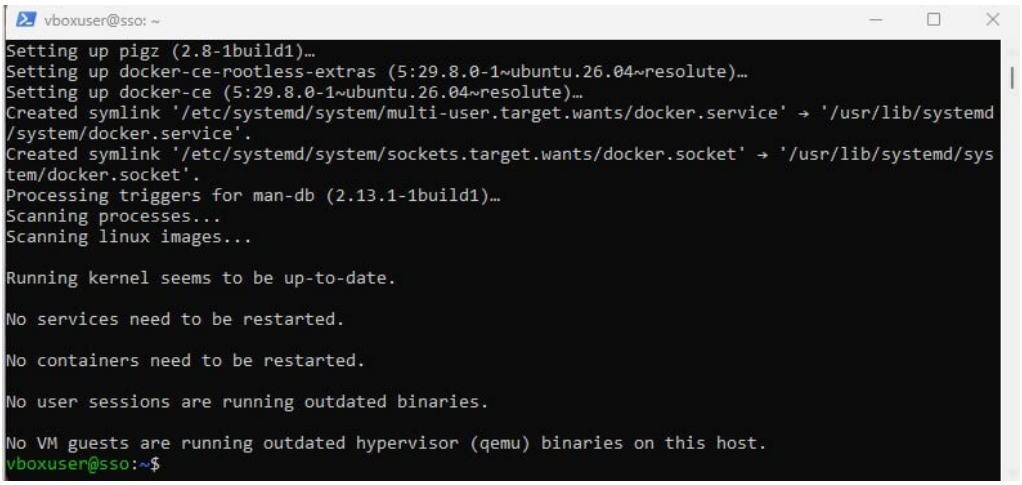


Figure 8 - Completion of the Docker Engine package installation

Docker Engine and its supporting components were installed successfully from the configured repository. The installation created the required systemd service and socket links, and the final package checks reported that the kernel was current and no service restart was required.

4.5 Verification of the Docker Service

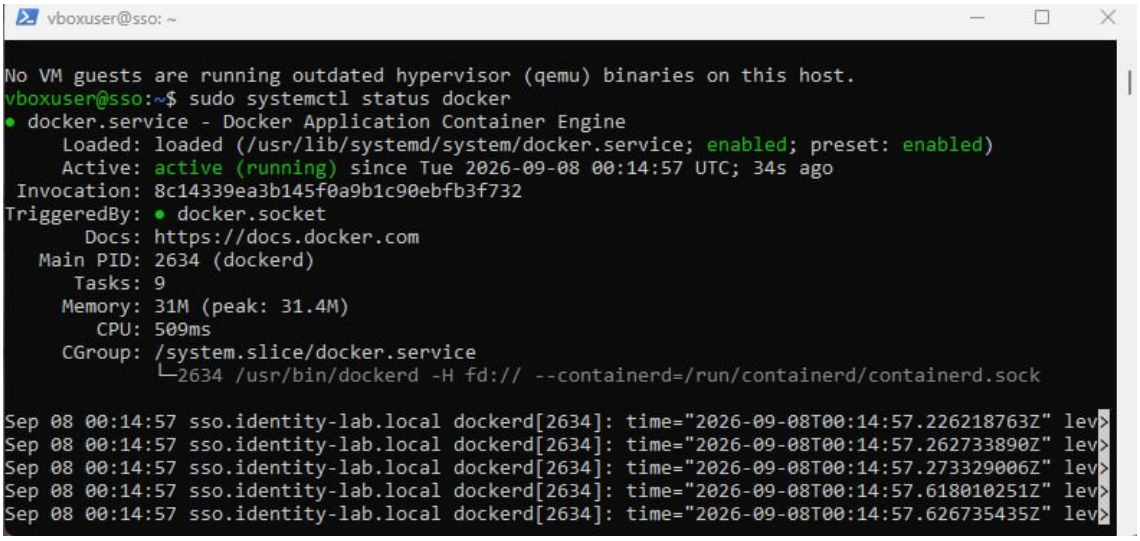


Figure 9 - Docker service running under systemd

The status of docker.service was checked with systemctl after installation. The service is loaded, enabled, and shown as active (running), confirming that the Docker daemon started successfully and is configured to start automatically with the server.

4.6 Verification of Docker and Docker Compose Versions

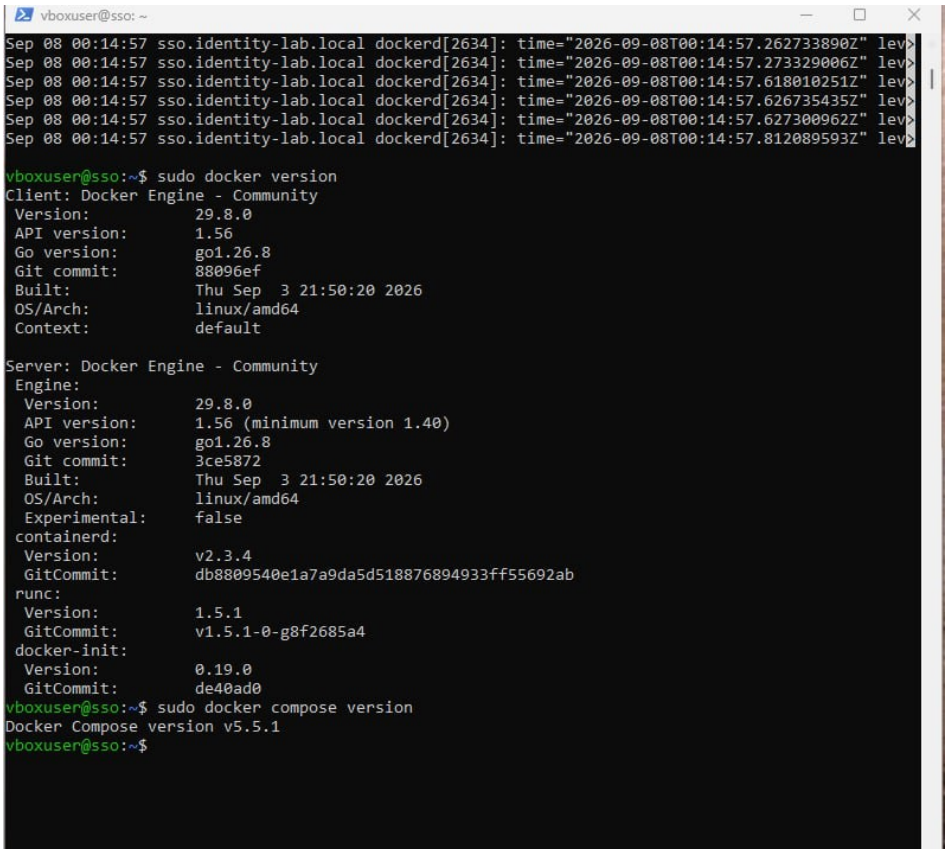


Figure 10 - Docker Engine client/server information and Docker Compose version

The docker version command confirmed that both the Docker client and server were available and communicating correctly, with Docker Engine Community version 29.8.0 displayed on the system. The docker compose version command also completed successfully and reported Docker Compose version v5.5.1.

4.7 Functional Test with the Hello-World Container

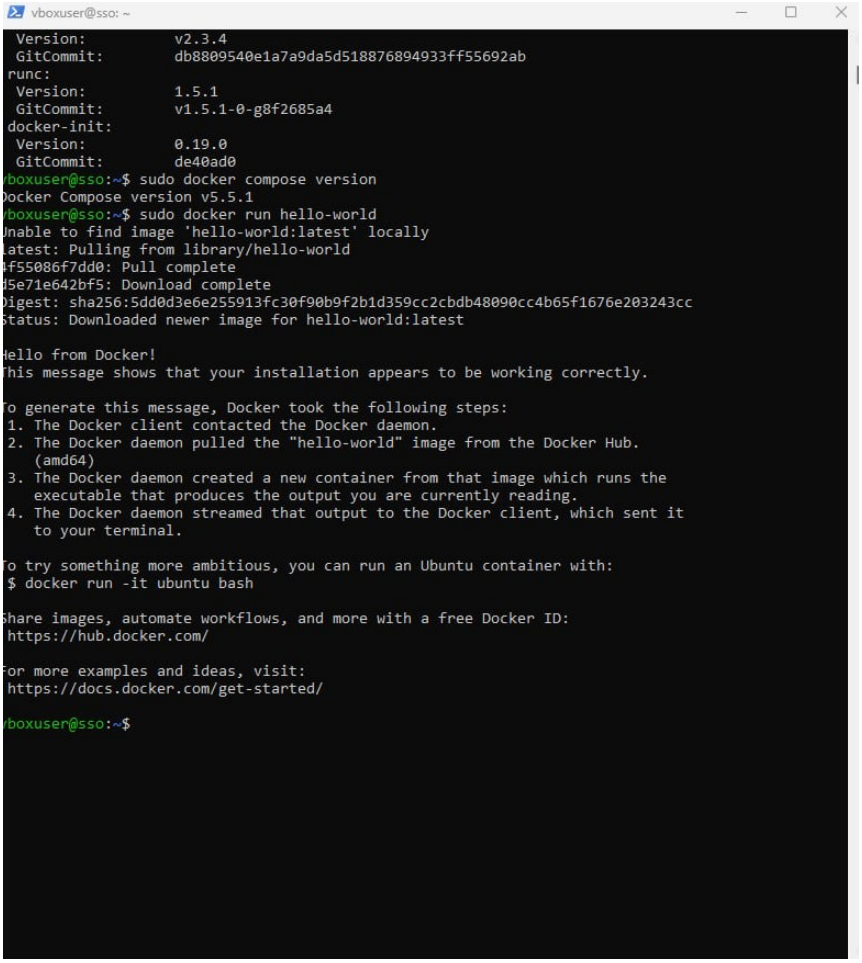


Figure 11 - Successful execution of the Docker hello-world container

The hello-world image was downloaded from Docker Hub and executed in a temporary test container. The message Hello from Docker! confirms that the client contacted the daemon, the image was pulled, the container was created, and its output was returned successfully.

4.8 Configuration of Docker Access Without sudo

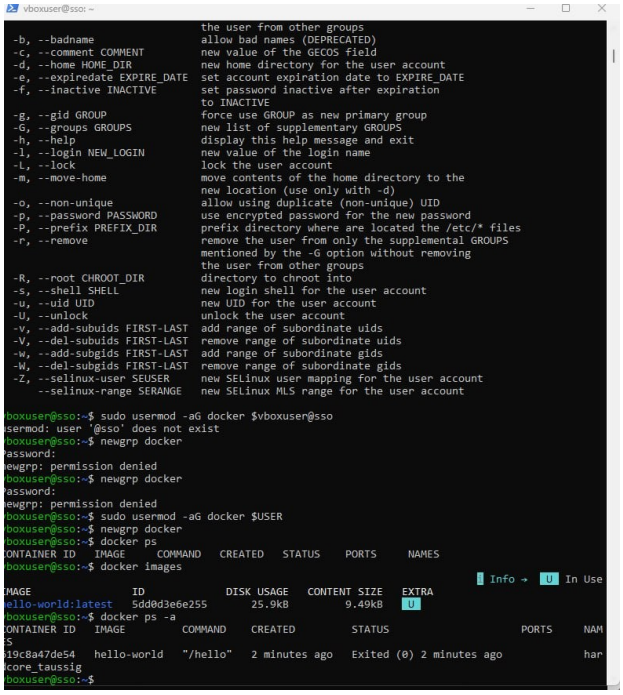


Figure 12 - Docker group membership and container checks without sudo

After an initial incorrect usermod command, the current account was correctly added to the docker group by using the $USER environment variable and the new group membership was activated with newgrp docker. The docker ps, docker images, and docker ps -a commands then ran without sudo; the hello-world image was present and its completed container correctly showed an Exited status.

5. Keycloak Deployment: Docker Network and PostgreSQL

A dedicated Docker bridge network and persistent storage volume were prepared for the Keycloak environment. PostgreSQL 17 was then deployed as the database service, connected to the private network, and tested for availability, connectivity, and data persistence after a container restart.

5.1 Creation of the Dedicated Docker Network

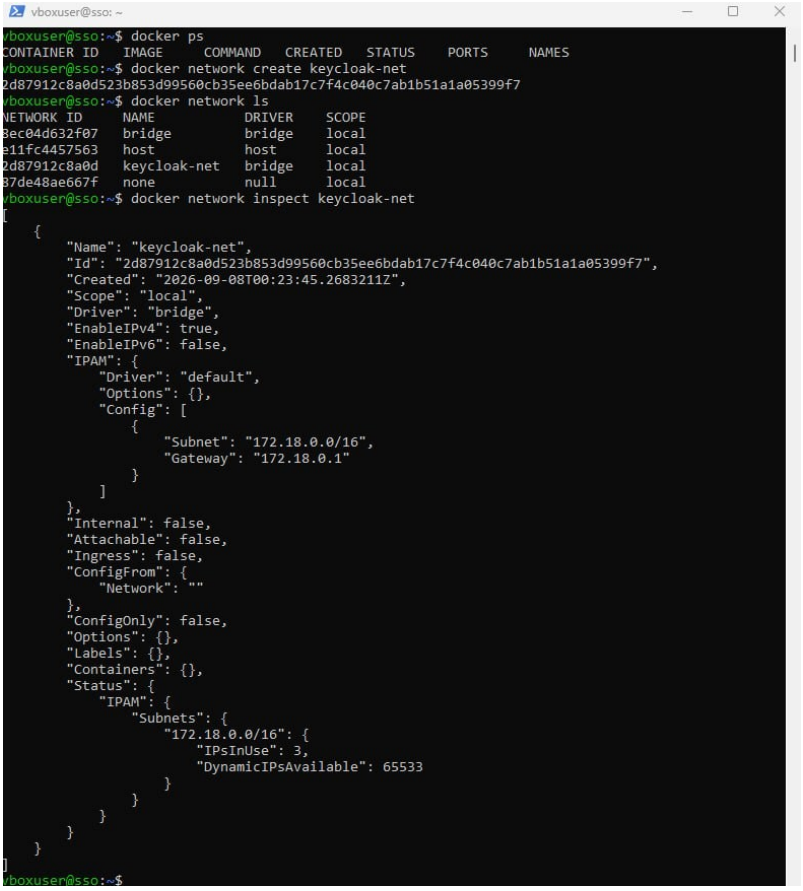


Figure 13 - Creation and inspection of the keycloak-net Docker network

A dedicated bridge network named keycloak-net was created to provide isolated communication between the PostgreSQL and Keycloak containers. The network listing and inspection output confirmed the bridge driver, the 172.18.0.0/16 subnet, and the 172.18.0.1 gateway; no containers were connected at this stage.

5.2 Creation of Persistent PostgreSQL Storage

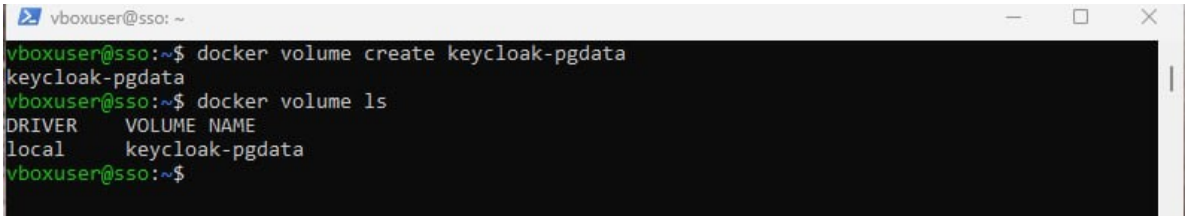


Figure 14 - Creation and listing of the keycloak-pgdata Docker volume

A named Docker volume called keycloak-pgdata was created for the PostgreSQL data directory. The docker volume ls output confirms that the volume exists and uses the local Docker volume driver, allowing database files to remain available independently of the container lifecycle.

5.3 Download of the PostgreSQL Container Image

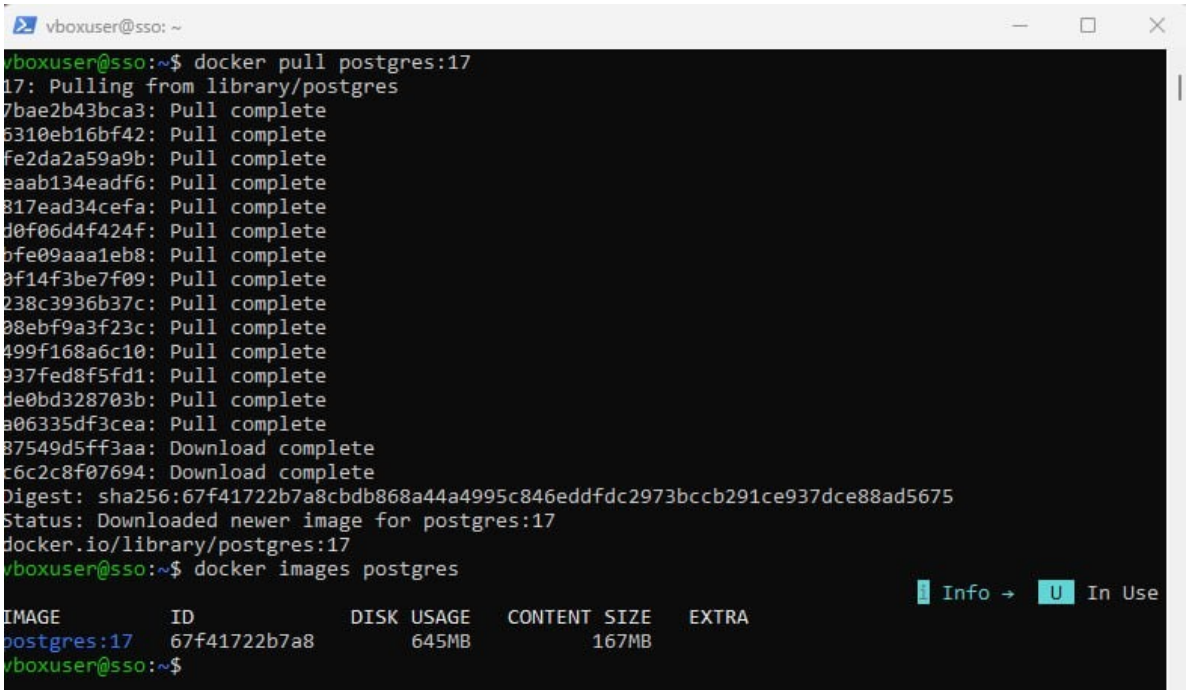


Figure 15 - PostgreSQL 17 image downloaded from Docker Hub

The official postgres:17 image was pulled from Docker Hub together with all required filesystem layers. A subsequent image listing verified that PostgreSQL 17 was stored locally and ready to be used for the database container.

5.4 Launch of the PostgreSQL Container

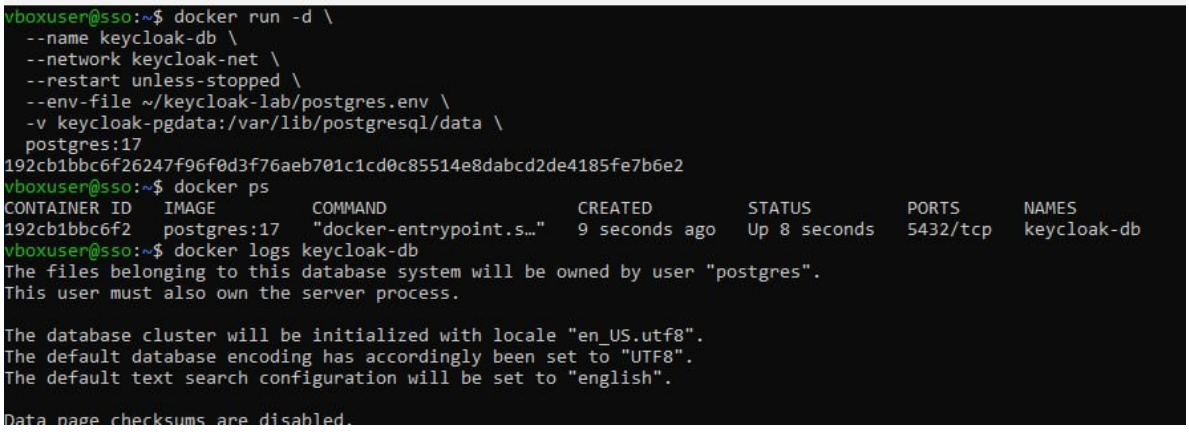


Figure 16 - PostgreSQL container launched as keycloak-db

The PostgreSQL container was started in detached mode with the name keycloak-db, attached to keycloak-net, and configured with the unless-stopped restart policy. Database settings were loaded from postgres.env, while keycloak-pgdata was mounted at /var/lib/postgresql/data; docker ps confirmed that the container was running and exposing its internal PostgreSQL port 5432.

5.5 Verification of PostgreSQL Startup Logs

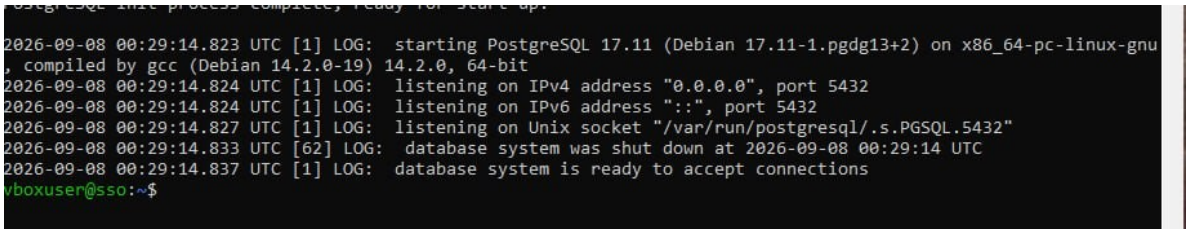


Figure 17 - PostgreSQL startup completed and listening on port 5432

The container logs show that PostgreSQL 17.11 started successfully and began listening on IPv4, IPv6, and the local Unix socket. The final message states that the database system is ready to accept connections, confirming successful initialization.

5.6 Database Connectivity and Network Verification

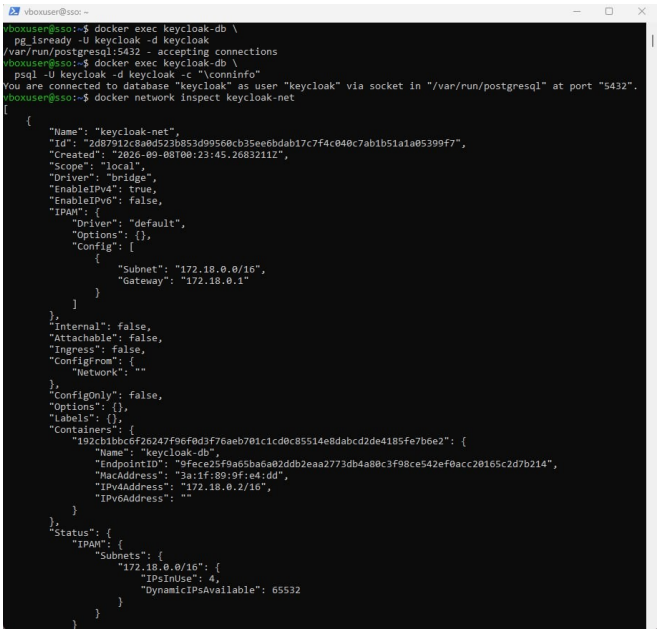


Figure 18 - PostgreSQL readiness, database connection, and network attachment checks

The pg_isready command confirmed that PostgreSQL was accepting connections, and psql successfully connected to the keycloak database as the keycloak user through the container socket on port 5432. Inspection of keycloak-net also showed keycloak-db attached to the network with the IPv4 address 172.18.0.2/16.

5.7 Verification of Persistent Storage After Restart

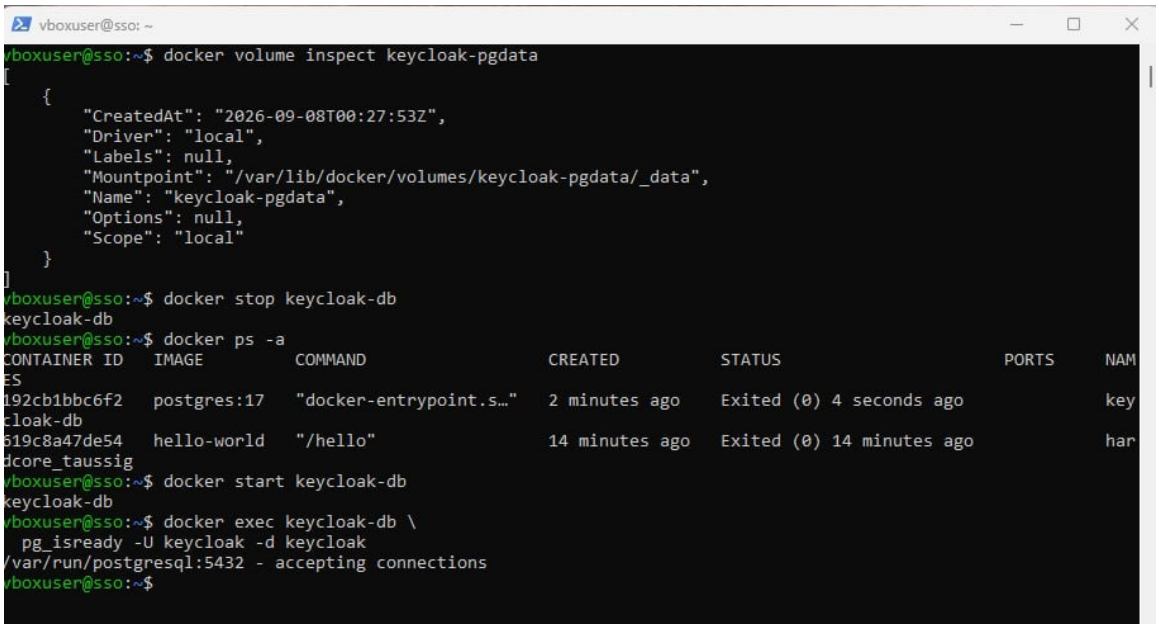


Figure 19 - PostgreSQL volume inspection and availability after a stop-start test

The keycloak-pgdata volume was inspected and its Docker-managed mount point was confirmed. The keycloak-db container was then stopped, observed with an Exited (0) status, started again, and retested with pg_isready; the successful response confirms that PostgreSQL returned to service while retaining its persistent volume.

6. Keycloak Container Deployment and PostgreSQL Integration

Keycloak was deployed as a Docker container on the existing keycloak-net network and configured through an environment file to use the PostgreSQL database. The deployment was then verified by checking both running containers, confirming creation of the Keycloak database schema, and testing the Keycloak HTTP endpoint from inside the Docker network.

6.1 Download and Launch of the Keycloak Container

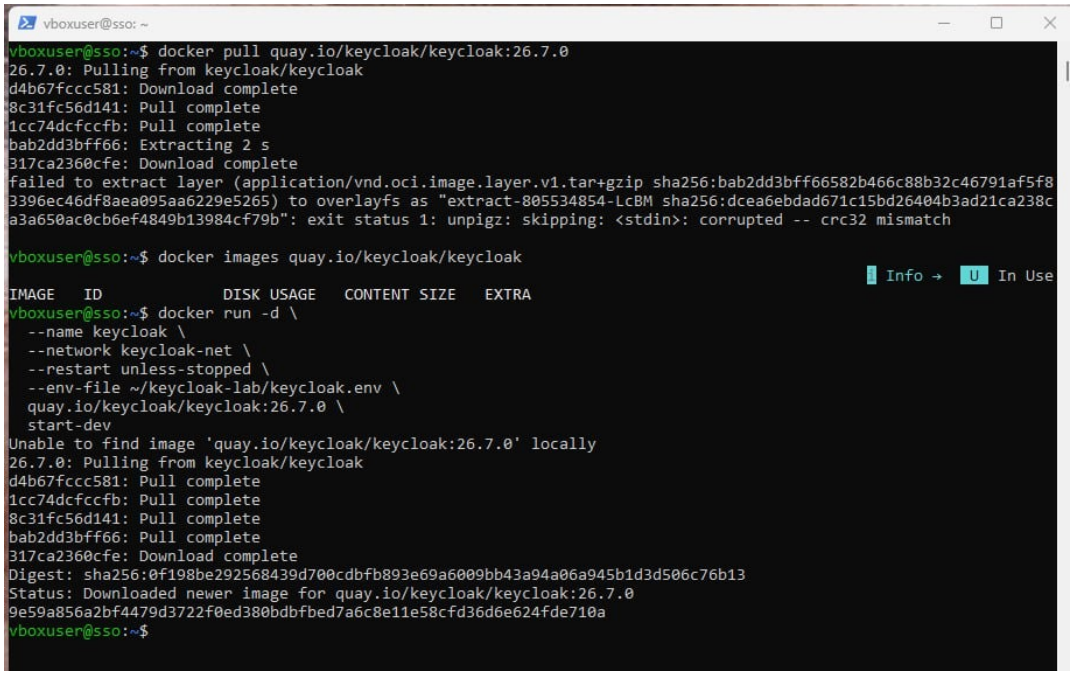


Figure 20 - Download and startup of the Keycloak 26.7.0 container

The initial standalone pull of quay.io/keycloak/keycloak:26.7.0 encountered a compressed-layer extraction error. The subsequent docker run command successfully downloaded the image again and launched the container as keycloak on keycloak-net, using the unless-stopped restart policy, the settings from keycloak.env, and the start-dev command.

6.2 Verification of Containers and Database Schema



Figure 21 - Running Keycloak and PostgreSQL containers with initialized Keycloak tables

The docker ps output confirms that both the keycloak container and the keycloak-db PostgreSQL container are running, with Keycloak exposing its internal ports 8080, 8443, and 9000. A psql table listing from the keycloak database shows Keycloak relations such as admin_event_entity, authentication_flow, and client, proving that the application connected to PostgreSQL and initialized its schema successfully.

6.3 Verification of the Keycloak HTTP Endpoint

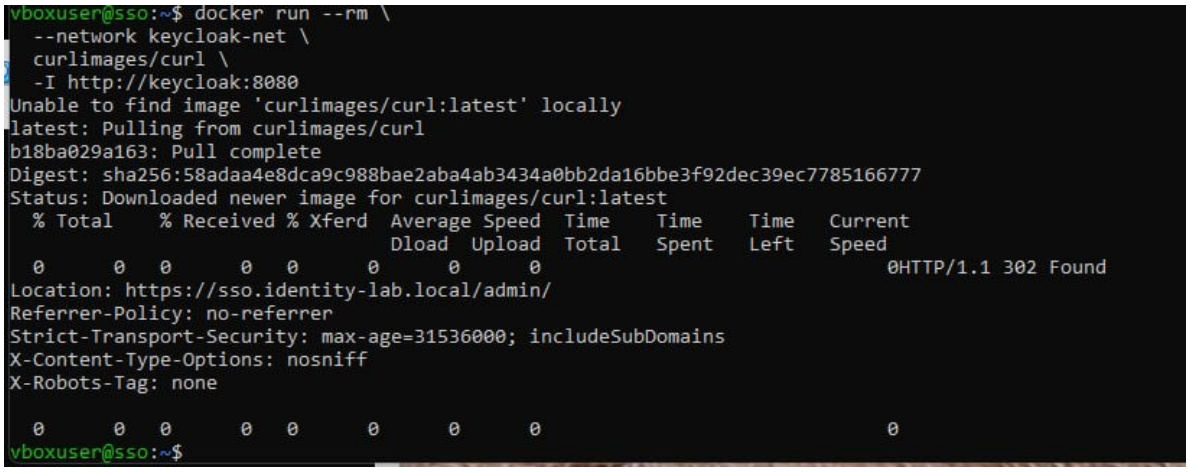


Figure 22 - HTTP response from Keycloak through the internal Docker network

A temporary curl container was attached to keycloak-net and used to request http://keycloak:8080. The HTTP/1.1 302 Found response redirected the request to https://sso.identity-lab.local/admin/, confirming Docker name resolution, network connectivity to the Keycloak service, and application-level handling of the configured external hostname.

7. Interim Result

The Keycloak 26.7.0 container is running on the dedicated Docker network and is successfully connected to PostgreSQL 17. The initialized Keycloak tables confirm working database integration, while the internal HTTP test confirms that the application is reachable and redirects requests to the configured HTTPS administration URL. The next stage is to configure the reverse proxy, TLS certificate, and browser access to the Keycloak administration console.